Practical No :1

Aim: Implement Breadth First Search Algorithm & Iterative Depth First Search Algorithm.

1. To implement the Breadth First Search algorithm to solve a given problem.
2. To implement the Iterative Depth First Search algorithm to solve the same problem.
3. Compare the performance and efficiency of both algorithms.

Program for BFS:

```
from collections import deque
def bfs(graph, start):
  visited = set()
  queue = deque([start])
  while queue:
    vertex = queue.popleft()
    if vertex not in visited:
      visited.add(vertex)
      print(vertex)
# Explore neighbors
      neighbors = graph[vertex]
      for neighbor in neighbors:
        if neighbor not in visited:
          queue.append(neighbor)
# Example usage
graph = {
'A': ['B', 'C'],
'B': ['A', 'D', 'E'],
'C': ['A', 'F'],
'D': ['B'],
'E': ['B', 'F'],
'F': ['C', 'E']
}
start_vertex = 'A'
bfs(graph, start_vertex)
```

Output:

```
A
B
C
D
E
F
```

Program for Iterative Depth First Search Algorithm:

```
from collections import defaultdict
class Graph:
  def __init__(self):
    self.graph = defaultdict(list)
  def add_edge(self, u, v):
    self.graph[u].append(v)
    self.graph[v].append(u) # Assuming an undirected graph
  def iterative_dfs(self, start, end):
    if start == end:
      return [start]
    visited = set()
    stack = [(start, [start])]
    while stack:
      current_vertex, path = stack.pop()
      visited.add(current_vertex)
      for neighbor in self.graph[current_vertex]:
        if neighbor not in visited:
          if neighbor == end:
            return path + [neighbor]
          stack.append((neighbor, path + [neighbor]))
    return None # No path found
# Example usage:
if __name__ == "__main__":
  g = Graph()
  g.add_edge(1, 2)
  g.add_edge(1, 3)
  g.add_edge(2, 4)
  g.add_edge(2, 5)
  g.add_edge(3, 6)
  g.add_edge(3, 7)
  g.add_edge(4, 8)
```

```
  g.add_edge(4, 9)
  g.add_edge(5, 10)
  g.add_edge(5, 11)
  g.add_edge(6, 12)
  g.add_edge(6, 13)
  g.add_edge(7, 14)
  g.add_edge(7, 15)
  start_node = 1
  end_node = 9
  shortest_path = g.iterative_dfs(start_node, end_node)
  if shortest_path:
    print(f"Shortest path from {start_node} to {end_node}:
{shortest_path}")
  else:
    print(f"No path found from {start_node} to {end_node}")
```

Output:

```
Shortest path from 1 to 9: [1, 2, 4, 9]
```

Practical No :2

Aim: A* Search and Recursive Best-First Search

1. Implement the A* Search algorithm for solving a path finding problem.
2. Implement the Recursive Best-First Search algorithm for the same problem.
3. Compare the performance and effectiveness of both algorithms.

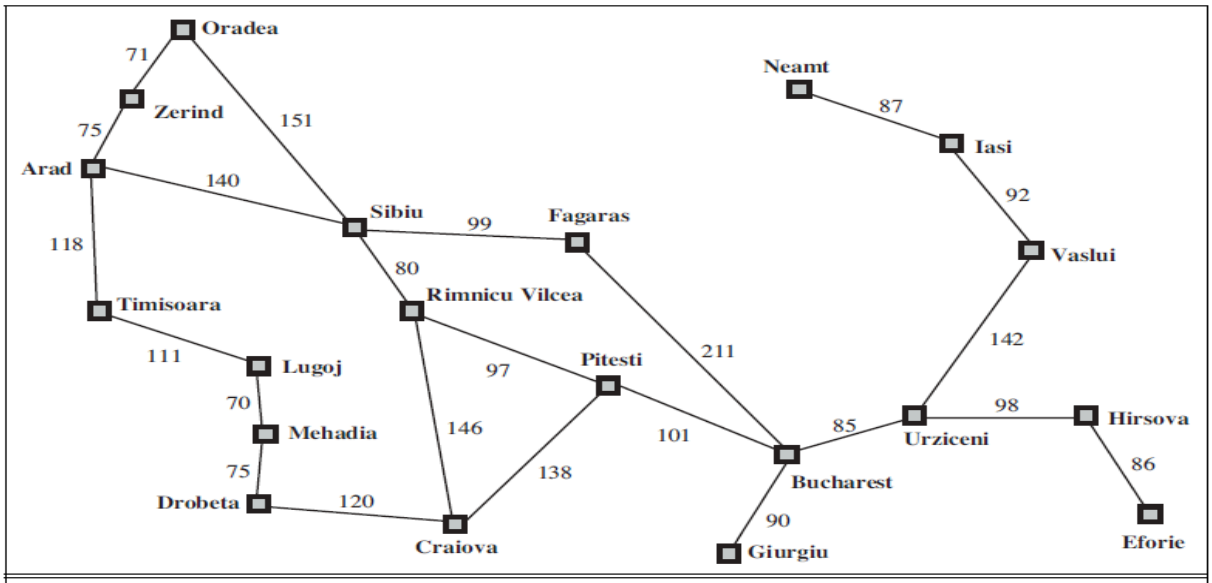


Figure 3.2 A simplified road map of part of Romania.

Program:

```
import heapq # For using a priority queue (min-heap)

# Romania Map: Each city maps to its neighbors and the distance to them
romania_map = {
  'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
  'Zerind': {'Arad': 75, 'Oradea': 71},
  'Oradea': {'Zerind': 71, 'Sibiu': 151},
  'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea':
80},
  'Timisoara': {'Arad': 118, 'Lugoj': 111},
  'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
  'Mehadia': {'Lugoj': 70, 'Dobreta': 75},
  'Dobreta': {'Mehadia': 75, 'Craiova': 120},
  'Craiova': {'Dobreta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
  'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
  'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
  'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
  'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni':
85},
  'Giurgiu': {'Bucharest': 90},
  'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
  'Hirsova': {'Urziceni': 98, 'Eforie': 86},
```

```
  'Eforie': {'Hirsova': 86},
  'Vaslui': {'Urziceni': 142, 'Iasi': 92},
  'Iasi': {'Vaslui': 92, 'Neamt': 87},
  'Neamt': {'Iasi': 87}
}

# Heuristic: Straight-line distance from each city to Bucharest
heuristic = {
  'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Dobreta': 242, 'Eforie':
161,
  'Fagaras': 178, 'Giurgiu': 77, 'Hirsova': 151, 'Iasi': 226, 'Lugoj':
244,
  'Mehadia': 241, 'Neamt': 234, 'Oradea': 380, 'Pitesti': 98, 'Rimnicu
Vilcea': 193,
  'Sibiu': 253, 'Timisoara': 329, 'Urziceni': 80, 'Vaslui': 199, 'Zerind':
374
}

# Node class to represent each city in the search
class Node:
  def __init__(self, city, parent=None, g=0, h=0):
    self.city = city # Current city name
    self.parent = parent # Parent node (used to trace back the path)
    self.g = g # Cost from start to this city
    self.h = h # Heuristic cost to goal
    self.f = g + h # Total cost = actual + estimated

  def __lt__(self, other):
    return self.f < other.f # Needed for heapq to compare nodes by f value

# A* search function
def a_star_search(start, goal):
  open_list = [] # Cities to explore (priority queue)
  heapq.heappush(open_list, Node(start, None, 0, heuristic[start])) # Add
start node
  closed_set = set() # Cities already explored

  while open_list:
    current = heapq.heappop(open_list) # Get the city with lowest f cost
```

```
    if current.city == goal: # If goal is reached
      path = []
      total_cost = current.g
      while current:
        path.append(current.city) # Trace back path from goal to start
        current = current.parent
      return path[::-1], total_cost # Return reversed path and total cost

    closed_set.add(current.city) # Mark current city as visited

    # Explore each neighbor of current city
    for neighbor, cost in romania_map[current.city].items():
      if neighbor in closed_set:
        continue # Skip if already visited

      g = current.g + cost # New cost from start to neighbor
      h = heuristic[neighbor] # Heuristic to goal from neighbor
      neighbor_node = Node(neighbor, current, g, h) # Create new node

      # Check if neighbor is already in open_list with better f
      skip = False
      for node in open_list:
        if node.city == neighbor and node.f <= neighbor_node.f:
          skip = True
          break

      if not skip:
        heapq.heappush(open_list, neighbor_node) # Add neighbor to queue

  return None, float('inf') # If goal not reachable

# Main program: Find path from Arad to Bucharest
if __name__ == "__main__":
  start_city = "Arad"
  goal_city = "Bucharest"
  path, cost = a_star_search(start_city, goal_city)

  if path:
    print("Path found:")
    print(" -> ".join(path)) # Display path step-by-step
```

```
    print("Total cost:", cost) # Display total distance
  else:
    print("No path found.")
```

Output:

```
Path found:
Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total cost: 418
```

Program for Recursive Best-First Search algorithm

```
import math
# Heuristic: straight-line distance to Bucharest (used as h(n))
heuristics = {
  'Arad': 366, 'Zerind': 374, 'Oradea': 380, 'Sibiu': 253, 'Timisoara':
329,
  'Lugoj': 244, 'Mehadia': 241, 'Drobeta': 242, 'Craiova': 160, 'Rimnicu
Vilcea':193,
  'Fagaras': 178, 'Pitesti': 98, 'Bucharest': 0, 'Giurgiu': 77,
'Urziceni': 80,
  'Hirsova': 151, 'Eforie': 161, 'Vaslui': 199, 'Iasi': 226, 'Neamt': 234
}
# Graph: roads with costs between cities
romania_map = {
  'Arad': {'Zerind': 75, 'Timisoara': 118, 'Sibiu': 140},
  'Zerind': {'Arad': 75, 'Oradea': 71},
  'Oradea': {'Zerind': 71, 'Sibiu': 151},
  'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea':
80},
  'Timisoara': {'Arad': 118, 'Lugoj': 111},
  'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
  'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
  'Drobeta': {'Mehadia': 75, 'Craiova': 120},
  'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
  'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
```

```
  'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
  'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
  'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni':
85},
  'Giurgiu': {'Bucharest': 90},
  'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
  'Hirsova': {'Urziceni': 98, 'Eforie': 86},
  'Eforie': {'Hirsova': 86},
  'Vaslui': {'Urziceni': 142, 'Iasi': 92},
  'Iasi': {'Vaslui': 92, 'Neamt': 87},
  'Neamt': {'Iasi': 87}
}
# Node class for RBFS
class Node:
  def __init__(self, city, parent=None, g=0):
    self.city = city
    self.parent = parent
    self.g = g # Cost from start to this node
    self.h = heuristics[city]
    self.f = max(self.g + self.h, getattr(parent, 'f', 0))
  # For comparison in sorting
  def __lt__(self, other):
      return self.f < other.f
# Recursive Best-First Search
def rbfs(node, goal, f_limit):
  print(f"Visiting {node.city}, f={node.f}, limit={f_limit}")
  if node.city == goal:
    return node, 0
  successors = []
  for neighbor, cost in romania_map[node.city].items():
    if node.parent and neighbor == node.parent.city:
      continue # Avoid going back to parent
    g = node.g + cost
    child = Node(neighbor, parent=node, g=g)
    successors.append(child)
  if not successors:
    return None, math.inf
  while True:
    successors.sort(key=lambda x: x.f)
    best = successors[0]
```

```
    if best.f > f_limit:
      return None, best.f
    alternative = successors[1].f if len(successors) > 1 else math.inf
    result, best.f = rbfs(best, goal, min(f_limit, alternative))
    if result is not None:
      return result, best.f
# Trace path from goal to start
def extract_path(node):
  path = []
  while node:
    path.append(node.city)
    node = node.parent
  return path[::-1]
# Main
if __name__ == "__main__":
  start = 'Arad'
  goal = 'Bucharest'
  start_node = Node(start)
  result, _ = rbfs(start_node, goal, math.inf)
  if result:
    path = extract_path(result)
    print("\nPath found:")
    print(" -> ".join(path))
    print("Total cost:", result.g)
  else:
    print("No path found.")
```

Output:

```
Visiting Arad, f=366, limit=inf
Visiting Sibiu, f=393, limit=447
Visiting Rimnicu Vilcea, f=413, limit=417
Visiting Pitesti, f=415, limit=417
Visiting Fagaras, f=417, limit=418
Visiting Rimnicu Vilcea, f=418, limit=447
Visiting Pitesti, f=418, limit=447
Visiting Bucharest, f=418, limit=447

Path found:
Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total cost: 418
```

Practical No :3

Aim: Decision Tree Learning

1. Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset
2. Evaluate the accuracy and effectiveness of the decision tree on test data
3. Visualize and interpret the generated decision tree.

Code:

```
# Import necessary libraries
import numpy as np
import pandas as pd  # Import Pandas for data loading
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the Iris dataset (iris.csv)
# Replace 'iris.csv' with the actual path to your Iris dataset file
data = pd.read_csv('/content/Iris.csv')

# Assuming the target variable is in a column named 'Target'
X = data.drop('Target', axis=1)
y = data['Target']

# Split the dataset into a training set and a testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Create a Decision Tree classifier
clf = DecisionTreeClassifier()

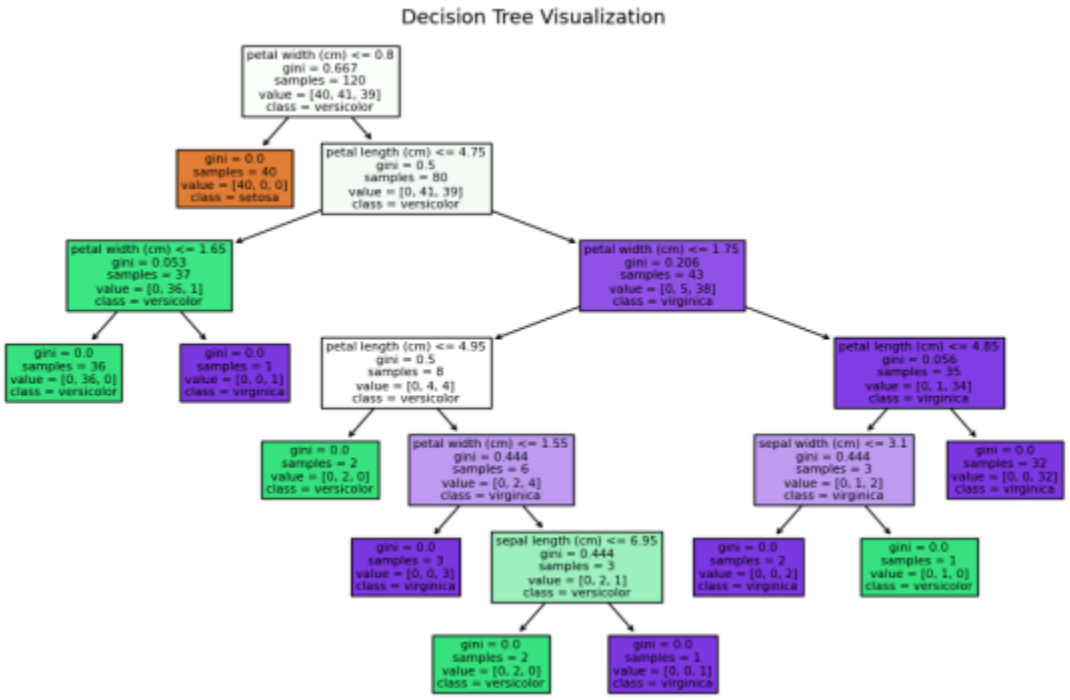
# Fit the classifier to the training data
clf.fit(X_train, y_train)

# Make predictions on the test data
y_pred = clf.predict(X_test)
```

```
# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Visualize and interpret the generated decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns,
class_names=data['Target'].unique())
plt.title("Decision Tree Visualization (Iris Dataset)")
plt.show()
```

Output:



Practical No :4

Aim: Feed Forward Back propagation Neural Network

1. Implement the Feed Forward Backpropagation algorithm to train a neural network.
2. Use a given dataset to train the neural network for a specific task.
3. Evaluate the performance of the trained network on test data

Program:

```
import numpy as np

# Define the sigmoid activation function and its derivative
def sigmoid(x):
  return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
  return x * (1 - x)

# Define the neural network class
class NeuralNetwork:
  def __init__(self, input_size, hidden_size, output_size):
    # Initialize weights with random values
    self.weights_input_hidden = np.random.uniform(-1, 1, (input_size,
hidden_size))
    self.weights_hidden_output = np.random.uniform(-1, 1, (hidden_size,
output_size))

  def forward(self, inputs):
    # Forward propagation
    self.hidden_input = np.dot(inputs, self.weights_input_hidden)
    self.hidden_output = sigmoid(self.hidden_input)
self.output_input = np.dot(self.hidden_output, self.weights_hidden_output)
    self.predicted_output = sigmoid(self.output_input)
    return self.predicted_output

  def backward(self, inputs, target, learning_rate):
    # Backpropagation
    error = target - self.predicted_output
    delta_output = error * sigmoid_derivative(self.predicted_output)
    error_hidden = delta_output.dot(self.weights_hidden_output.T)
    delta_hidden = error_hidden * sigmoid_derivative(self.hidden_output)

    # Update weights
    self.weights_hidden_output += np.outer(self.hidden_output,
delta_output) * learning_rate
    self.weights_input_hidden += np.outer(inputs, delta_hidden) *
learning_rate
```

```
  def train(self, training_data, targets, epochs, learning_rate):
    for epoch in range(epochs):
      for i in range(len(training_data)):
        inputs = training_data[i]
        target = targets[i]
        self.forward(inputs)
        self.backward(inputs, target, learning_rate)

  def predict(self, inputs):
    return self.forward(inputs)

# Define XOR dataset
training_data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
targets = np.array([[0], [1], [1], [0]])

# Create and train the neural network
input_size = 2
hidden_size = 4
output_size = 1
learning_rate = 0.1
epochs = 10000

nn = NeuralNetwork(input_size, hidden_size, output_size)
nn.train(training_data, targets, epochs, learning_rate)

# Test the trained network
for i in range(len(training_data)):
  inputs = training_data[i]
  prediction = nn.predict(inputs)
  print(f"Input: {inputs}, Predicted Output: {prediction}")
```

Output:

```
Input: [0 0], Predicted Output: [0.08926787]
Input: [0 1], Predicted Output: [0.92655649]
Input: [1 0], Predicted Output: [0.91639718]
Input: [1 1], Predicted Output: [0.07837027]
```

Practical No :5

Aim: Support Vector Machines (SVM)

1.Implement the SVM algorithm for binary classification.

2.Train an SVM model using a given dataset and optimize its parameters.

3. Evaluate the performance on test data and analyze the results.

Program:

```
import numpy as np
import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset
data = pd.read_csv("D:\\TYCS_DS\\Dataset\\Iris.csv")

# Display first few rows
print("Dataset Preview:")
print(data.head())

# Features and Target
X = data.drop('Target', axis=1)
y = data['Target']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create SVM classifier
svm_classifier = SVC(kernel='linear')
```

```
# Train model
svm_classifier.fit(X_train, y_train)

# Predict test data
y_pred = svm_classifier.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
========================== RESTART: D:/TYCS_IT/p1.py =====
Dataset Preview:
   ID  SepalLength Cm  ...  PetalWidth Cm       Target
0   1             5.1  ...            0.2  Iris-setosa
1   2             4.9  ...            0.2  Iris-setosa
2   3             4.7  ...            0.2  Iris-setosa
3   4             4.6  ...            0.2  Iris-setosa
4   5             5.0  ...            0.2  Iris-setosa

[5 rows x 6 columns]

Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

Program:

```
# Import required libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset
data = load_breast_cancer()

# Features and Target
X = data.data
y = data.target
```

```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create SVM classifier
svm_model = SVC(kernel='linear')

# Train the model
svm_model.fit(X_train, y_train)

# Make predictions
y_pred = svm_model.predict(X_test)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
========================== RESTART: D:/TYCS_IT/p1.py ===
Accuracy: 0.956140350877193

Confusion Matrix:
[[41  2]
 [ 3 68]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.95      0.94        43
           1       0.97      0.96      0.96        71

    accuracy                           0.96       114
   macro avg       0.95      0.96      0.95       114
weighted avg       0.96      0.96      0.96       114
```

Practical No :6

Aim: Adaboost Ensemble Learning

1.Implement the adaboost algorithm to create an ensemble of weak classifiers.

2.Train the ensemble model on a given dataset and evaluate its performance.

3.Compare the results with individual weak classifier.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Dataset
data = pd.read_csv("D:\\TYCS_DS\\Dataset\\Iris.csv")
```

```
# Features and Target
X = data.drop('Target', axis=1)
y = data['Target']

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42)

# Weak Classifier (Decision Tree Stump)
weak_classifier = DecisionTreeClassifier(max_depth=1)

weak_classifier.fit(X_train, y_train)

weak_pred = weak_classifier.predict(X_test)

weak_accuracy = accuracy_score(y_test, weak_pred)

print("Weak Classifier Accuracy:")
print(weak_accuracy)

# AdaBoost Ensemble Model

adaboost_model = AdaBoostClassifier(
   estimator=DecisionTreeClassifier(max_depth=1),
   n_estimators=50,
   random_state=42
)

adaboost_model.fit(X_train, y_train)

ada_pred = adaboost_model.predict(X_test)

ada_accuracy = accuracy_score(y_test, ada_pred)
```

```
print("\nAdaBoost Accuracy:")
print(ada_accuracy)

print("\nClassification Report (AdaBoost):")
print(classification_report(y_test, ada_pred))

# Performance Comparison
print("\nPerformance Comparison")
print("----------------------")
print("Weak Classifier Accuracy :", weak_accuracy)
print("AdaBoost Accuracy       :", ada_accuracy)

# Plot Accuracy Comparison Graph
models = ['Weak Classifier', 'AdaBoost']
accuracies = [weak_accuracy, ada_accuracy]

plt.figure(figsize=(6, 4))
plt.bar(models, accuracies)

plt.title('Accuracy Comparison: Weak Classifier vs AdaBoost')
plt.xlabel('Models')
plt.ylabel('Accuracy')
plt.ylim(0, 1.1)

# Display accuracy values on bars
for i, value in enumerate(accuracies):
    plt.text(i, value + 0.02, f'{value:.2f}', ha='center')

plt.show()
```

Output:

```
Weak Classifier Accuracy:
0.6333333333333333

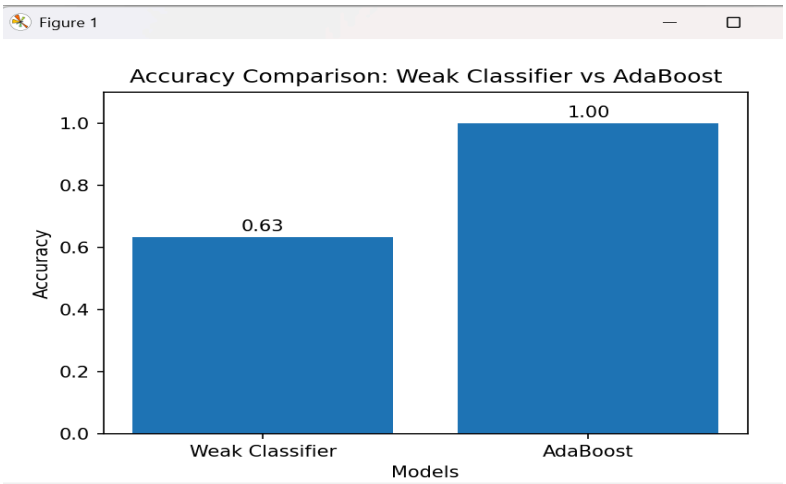
AdaBoost Accuracy:
1.0

Classification Report (AdaBoost):
                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30


Performance Comparison
----------------------
Weak Classifier Accuracy : 0.6333333333333333
AdaBoost Accuracy        : 1.0
```



Practical No :7

Aim: Naïve Bayes Classifier

1. To implement the Naive Bayes algorithm for classification.
2. Train a Naive Bayes model using a given dataset and calculate class probabilities.
3. Evaluate the accuracy of the model on test data and analyze the results.

Program:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load Iris Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Naive Bayes Classifier
clf = GaussianNB()

# Train Model
clf.fit(X_train, y_train)

# Predict Test Data
y_pred = clf.predict(X_test)

# Calculate Class Probabilities
probabilities = clf.predict_proba(X_test)

print("Class Probabilities for first 5 test samples:")
print(probabilities[:5])
```

```
# Model Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
============================ RESTART: D:/TYCS_IT/p1.py ==
Class Probabilities for first 5 test samples:
[[5.97327448e-090 9.95635767e-001 4.36423302e-003]
 [1.00000000e+000 4.96158126e-014 6.54922363e-021]
 [7.31890302e-290 4.92947614e-012 1.00000000e+000]
 [2.81842533e-094 9.77593559e-001 2.24064412e-002]
 [1.13877801e-105 8.70022596e-001 1.29977404e-001]]

Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Program:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score
import pandas as pd
# Load the football dataset from CSV
dataset = pd.read_csv('/content/argfrc_dataset.csv')
# Separate features (X) and labels (y)
X = dataset[['Argentina', 'France']].values
y = dataset['
'].values
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Create a Naïve Bayes classifier (Gaussian Naïve Bayes for continuous
features)
clf = GaussianNB()
# Train the classifier on the training data
clf.fit(X_train, y_train)
# Make predictions on the test data
y_pred = clf.predict(X_test)
# Calculate and print the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output:

```
Accuracy: 0.3333333333333333
```

Practical No :8

Aim: K - Nearest Neighbors (K-NN)

1. Implement the K-NN algorithm for classification or regression.
2. Apply the K-NN algorithm to a given dataset and predict the class or value for test data.
3. Evaluate the accuracy or error of the predictions and analyze the results

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load Iris Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create K-NN Classifier
knn = KNeighborsClassifier(n_neighbors=5)

# Train the Model
knn.fit(X_train, y_train)

# Predict Test Data
y_pred = knn.predict(X_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Predict Class for First Test Sample
sample_prediction = knn.predict([X_test[0]])
print("\nPrediction for First Test Sample:", sample_prediction[0])

# ----------------------------
# Accuracy Graph
# ----------------------------
plt.figure(figsize=(5,4))
plt.bar(['K-NN'], [accuracy])

plt.title('K-NN Classification Accuracy')
plt.ylabel('Accuracy')
plt.ylim(0, 1.1)

plt.text(0, accuracy + 0.02, f'{accuracy:.2f}', ha='center')

plt.show()
```

Output:

```
========== RESTART: D:/TYCS_IT/p1.py ==
Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


Prediction for First Test Sample: 1
```

Practical No :9

Aim: Association Rule Mining

1. Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent item sets.
2. Generate association rules from the frequent itemsets and calculate their support and confidence.
3. Interpret and analyze the discovered association rules.

Program:

```
!pip install mlxtend
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
import pandas as pd
dataset =[
['milk','bread','nuts'],
['milk','bread'],
['milk','eggs','nuts'],
['milk','bread','eggs'],
['bread','nuts']
]
df = pd.DataFrame(dataset)
df_encoded = pd.get_dummies(df,prefix='',prefix_sep='')
frequent_itemsets = apriori(df_encoded,min_support=0.6,use_colnames=True)
print("Frequent Itemsets:")
```

```
print(frequent_itemsets)
rules =
association_rules(frequent_itemsets,metric='confidence',min_threshold=0.7)
print("\nAssociation Rules:")
print(rules)
```

Output:

```
   support       itemsets
0      0.8         (milk)
1      0.6        (bread)
2      0.6  (bread, milk)

Association Rules:
  antecedents consequents  antecedent support  consequent support  support  \
0     (bread)      (milk)                 0.6                 0.8      0.6
1      (milk)     (bread)                 0.8                 0.6      0.6

   confidence  lift  representativity  leverage  conviction  zhangs_metric  \
0        1.00  1.25               1.0      0.12         inf            0.5
1        0.75  1.25               1.0      0.12         1.6            1.0

   jaccard  certainty  kulczynski
0     0.75      1.000       0.875
1     0.75      0.375       0.875
```

Practical No :10

Aim: Demo of open AI and TensorFlow Tools

1. Explore and experiment with OpenAI or TensorFlow tools and libraries.
2. Perform a demonstration or mini-project showcasing the capabilities of the tools.
3. Discuss and present the findings and potential applications.

Practical:

```
# Import Libraries
import tensorflow as tf
from tensorflow.keras.datasets import mnist
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load MNIST Dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize Data
X_train = X_train / 255.0
X_test = X_test / 255.0

# Create Neural Network Model
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train Model
model.fit(X_train, y_train, epochs=5)

# Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy)
```

```
# Predict First Test Image
prediction = model.predict(X_test[:1])

print("Predicted Digit:", prediction.argmax())

# Display Image
plt.imshow(X_test[0], cmap='gray')
plt.title("Test Image")
plt.axis('off')
plt.show()
```

Output:

```
Predicted Digit: 7
```

